Phase 1: Prerequisites

Complete these one-time setup tasks before configuring your management platform (Panorama or SCM)

This document covers all prerequisite steps for deploying VM-Series firewalls on Azure. Complete each step in order, then use the checklist at the end to confirm everything is in place before moving to Phase 2.

1.1 Activate Software NGFW Credits

Palo Alto Docs: Activate Credits

Before creating deployment profiles or licensing firewalls, you need an active credit pool.

1. Log in to the Customer Support Portal.
2. Navigate to **Products > Software NGFW Credits**.
3. Locate your credit pool authorization code (from your purchase order).
4. Click **Activate** and enter the auth code to fund the credit pool.
5. Verify the credit pool shows available credits for your desired VM-Series model and subscription tier.

> **NOTE**
> Credit pools are shared across your account. Each firewall deployment consumes credits based on the vCPU count and subscriptions defined in the deployment profile.

1.2 Verify Strata Cloud Manager (SCM)

Strata Cloud Manager provides centralized management for your Palo Alto Networks deployment. It must be provisioned before deployment.

1. Log in to Strata Cloud Manager.
2. Verify your tenant is active and you can access the management console.
3. Under **Products** for your tenant, confirm the following are listed:
    - Strata Cloud Manager
    - Strata Logging Service
    - Telemetry

> **NOTE**
> NGFW Management is enabled by default on new tenants — no separate flag request is required.

1.3 Verify Strata Logging Service (SLS) — Optional

Strata Logging Service provides centralized log collection and analysis. It must be provisioned before firewalls can forward logs.

1. In Strata Cloud Manager, navigate to **Settings > Strata Logging Service**.
2. Verify the logging service is active and your log storage allocation is visible.

1.4 Create a Deployment Profile

Palo Alto Docs: Create a Deployment Profile | Manage a Deployment Profile

The deployment profile defines the firewall model, vCPU allocation, and subscriptions that each bootstrapped firewall will receive.

1. In the Customer Support Portal, go to **Products > Software NGFW Credits**.
2. Select your credit pool and click **Create Deployment Profile**.
3. Configure: **Profile Name**, **Firewall Model**, **vCPU Count**, **Subscriptions**, **Token Count**.
4. Click **Create**. Note the generated **auth code** (format: `D1234567`).

> **PROFILES PARTITION BY LICENSING TIER AND VCPU, NOT BY TOPOLOGY**
>
> A deployment profile is a license SKU bundle — firewall model, vCPU count, and subscription mix (Threat Prevention, WildFire, URL Filtering, etc.). The **auth code** the profile generates is what each bootstrapping VM-Series presents to license itself.
>
> Rule of thumb: **one profile per distinct (vCPU count + subscription bundle)**. A few common patterns:
>
> - All firewalls identical → one profile, one auth code.
> - Inbound at 4 vCPU, OBEW at 8 vCPU → two profiles (different vCPU counts).
> - Same shape across AWS, Azure, and GCP → one profile shared across all three clouds.

1.5 Obtain the Auth Code

1. In the Customer Support Portal, go to **Products > Software NGFW Credits > Deployment Profiles**.
2. Locate your deployment profile and copy the **Auth Code**.
3. You will use this code when creating the Bootstrap Definition in Phase 2.

1.6 Generate Auto-Registration PIN ID and PIN Value

Palo Alto Docs: Install a Device Certificate on the VM-Series Firewall

1. Log in to the Customer Support Portal.
2. Navigate to **Products > VM-Series Auth-Codes**.
3. Click **Create Registration PIN**.
4. Copy both: **PIN ID** (UUID format) and **PIN Value** (hex string).

> **NOTE**
> Without a valid PIN, the firewall cannot fetch a device certificate and will fail to authenticate with the licensing infrastructure.

1.7 Accept the Azure Marketplace EULA for VM-Series

You must accept the Azure Marketplace terms before deploying VM-Series. This is a one-time action per Azure subscription per product offering.

Option A: Azure CLI (Fastest)

Accept the terms for the `vmseries-flex` offer with the plan that matches your license model:

```
# BYOL
az vm image terms accept \
  --publisher paloaltonetworks \
  --offer vmseries-flex \
  --plan byol

# Bundle 1 (PAYG)
az vm image terms accept \
  --publisher paloaltonetworks \
  --offer vmseries-flex \
  --plan bundle1

# Bundle 2 (PAYG)
az vm image terms accept \
  --publisher paloaltonetworks \
  --offer vmseries-flex \
  --plan bundle2
```

Option B: Azure Portal

1. Log in to the Azure Portal and open **Marketplace**.
2. Search for **Palo Alto Networks VM-Series Next Generation Firewall** and select the offer that matches your license model.
3. Click **Get It Now**, review the terms, and click **Continue** to accept the EULA.

VERIFICATION

Confirm that the marketplace terms are accepted for your subscription:

```
az vm image terms show \
  --publisher paloaltonetworks \
  --offer vmseries-flex \
  --plan byol \
  --query 'accepted'
```

Returns `true` if accepted, `false` otherwise.

1.8 Azure Subscription, RBAC & Service Principal for Terraform

Terraform requires an Azure identity with sufficient privileges to provision all resources (resource groups, VNets, NSGs, route tables, load balancers, public IPs, VMs, storage accounts).

Option A: Logged-in user (development / Cloud Shell)

The simplest path for development is to authenticate Terraform with your own user account via `az login`. The user must have **Contributor** on the target subscription.

```
az login
az account set --subscription "<your-subscription-id>"
az account show --query '{subscription:name, id:id, tenantId:tenan
```

Option B: Service Principal (Preferred for CI/CD)

Create a Service Principal scoped to the target subscription for automation pipelines.

```
# Capture your subscription ID first
SUBSCRIPTION_ID=$(az account show --query id -o tsv)

# Create the SP with Contributor on the subscription
az ad sp create-for-rbac \
  --name "terraform-vmseries" \
  --role "Contributor" \
  --scopes "/subscriptions/${SUBSCRIPTION_ID}"
```

Export the credentials as environment variables:

```
export ARM_CLIENT_ID="<appId>"
export ARM_CLIENT_SECRET="<password>"
export ARM_TENANT_ID="<tenant>"
export ARM_SUBSCRIPTION_ID="<subscriptionId>"
```

Verify vCPU Quota in Your Region

VM-Series typically runs on **Standard_DS3_v2** or higher (4 vCPUs each). Confirm you have quota:

```
# Replace 'eastus' with your target region
az vm list-usage --location eastus --output table | grep -i 'stand
```

IMPORTANT

The initial deployment requires **Contributor or equivalent** on the subscription. If Terraform also needs to create role assignments, add **User Access Administrator** as well. Scope can be reduced post-deployment.

1.9 Deployment Environment & Tooling

Set up one of the following environments before starting Phase 4 (Terraform deployment).

Option A: Azure Cloud Shell (Quickest)

Cloud Shell ships with Terraform and the Azure CLI pre-installed. It automatically authenticates as the logged-in portal user.

1. Open **Cloud Shell** from the Azure Portal (top-right toolbar). Choose **Bash**.
2. Confirm the bundled tools:

```
terraform -v
az --version
git --version
az account show
```

Option B: Local Workstation or Bastion Host

TOOL	PURPOSE	INSTALL
Terraform CLI	Infrastructure provisioning	developer.hashicorp.com/terraform/downloads
Azure CLI	Azure authentication & resource management	learn.microsoft.com/cli/azure/install-azure-cli
Git	Version control for Terraform code	git-scm.com/downloads
VS Code (recommended)	HCL syntax highlighting	code.visualstudio.com

VERIFICATION

Confirm all tools are installed and your Azure identity is correct:

```
terraform -v
az --version
git --version
az account show
```

WINDOWS USERS

All shell commands in this guide use Bash syntax. If deploying from Windows, use **WSL 2** (recommended), **Git Bash**, or translate commands for PowerShell.

1.10 Terraform Remote State Backend

A remote state backend is critical for team collaboration and preventing concurrent modifications. Azure's `azurerm` backend uses a Storage Account container with native blob lease locking.

Create the Backend Resources

```
RG=tfstate-rg
LOCATION=eastus
SA=<your-unique-storage-account-name>
CONTAINER=tfstate

# Resource group
az group create --name $RG --location $LOCATION

# Storage account (versioned, TLS 1.2 only)
az storage account create \
  --name $SA \
  --resource-group $RG \
  --location $LOCATION \
  --sku Standard_LRS \
  --kind StorageV2 \
  --min-tls-version TLS1_2 \
  --allow-blob-public-access false

# Enable blob versioning for state history
az storage account blob-service-properties update \
  --account-name $SA \
  --resource-group $RG \
  --enable-versioning true

# Container for state
az storage container create --name $CONTAINER --account-name $SA
```

Configure the Backend (backend.tf)

```
terraform {
  backend "azurerm" {
    resource_group_name  = "tfstate-rg"
    storage_account_name = "<your-unique-storage-account-name>"
    container_name       = "tfstate"
    key                  = "pan-vmseries/azure/terraform.tfstate"
```

```
  }
}
```

> **NOTE**
>
> Place the state Storage Account in the **same region** as your deployment to minimize latency. Blob versioning provides state history; the native blob lease handles concurrent-modification locking automatically.

1.11 Git Repository for Terraform Code

Store your Terraform configuration in a remote Git repository for version control and collaboration.

1. Create a new repository for your PAN deployment code.
2. Clone it to your deployment machine or Cloud Shell session.
3. Bootstrap your repo from the official Palo Alto Networks reference modules: `PaloAltoNetworks/terraform-azurerm-swfw-modules`.
4. For advanced deployments, configure a CI/CD pipeline that authenticates to Azure (via the Service Principal from 1.8) and runs `terraform plan` / `apply`.

Post-Prerequisites Checklist

Confirm every item below before moving to Phase 2 (Management Platform Configuration). Check each box as you complete it.

LICENSING & CREDITS

- [ ] Software NGFW credit pool is activated and shows available credits
- [ ] Deployment profile created with correct firewall model, vCPU count, and subscriptions
- [ ] Auth code copied from deployment profile (format: `D1234567`)

DEVICE CERTIFICATES

- [ ] Auto-registration PIN ID generated (UUID format)
- [ ] Auto-registration PIN Value generated (hex string)

STRATA CLOUD MANAGER

- [ ] SCM tenant is active and accessible
- [ ] Strata Logging Service is provisioned (if required)

AZURE SUBSCRIPTION

- [ ] Azure Marketplace EULA accepted for VM-Series (`az vm image terms show` returns `true`)
- [ ] Azure identity configured (logged-in user *or* Service Principal with Contributor role)
- [ ] vCPU quota verified in target region for required VM family

TOOLING & INFRASTRUCTURE

- [ ] Terraform CLI installed and verified (`terraform -v`)
- [ ] Azure CLI installed and authenticated (`az account show`)
- [ ] Git installed (`git --version`)

- [ ] Remote state Storage Account created with blob versioning enabled
- [ ] `backend.tf` configured with correct storage account, container, and state key
- [ ] Git repository created and cloned for Terraform code

VALUES TO CARRY FORWARD

Record these values — you will need them in Phase 2 and Phase 4.

VALUE	SOURCE	YOUR VALUE
Deployment profile auth code	Step 1.5	
Auto-registration PIN ID	Step 1.6	
Auto-registration PIN Value	Step 1.6	
Azure Subscription ID	Step 1.8	
Azure Tenant ID	Step 1.8	
Service Principal App ID	Step 1.8 (if using SP)	
Terraform state Storage Account name	Step 1.10	
Target Azure region	Step 1.8	

READY FOR PHASE 2

Once every box is checked and the values table is filled in, proceed to **Phase 2: Management Platform Configuration** in the full deployment guide.